# 1 Agile Fundamentals

https://www.geeksforgeeks.org/software-testing/what-is-agile-methodology/

## 1.1 What Does It Mean to Be Agile?

Being agile does not mean following a strict process, but rather a way of thinking that is based on continuous learning, experimentation, and customer collaboration. The slides stress that agile teams are small, cross-functional, and fully dedicated to a product. They divide complex issues into small pieces and deliver functional software in short iterations called sprints. Daily stand-up meetings make progress transparent and facilitate the removal of obstacles. The aim is to quickly learn from feedback and change direction, rather than following a plan because it was agreed upon in advance.

Agile is particularly useful when faced with complex issues, where the solution is unknown, and changes in requirements are to be expected. It is not applicable for simple, repetitive work such as managing a factory production line. Real agility lies in a balance between experimentation and accountability, where you maintain clear goals and profit-and-loss responsibility but empower teams to find out what works best to achieve them.

## 1.2 When Agile Works (and When It Doesn't)

The Stacey complexity matrix is a graph of requirement uncertainty versus technological uncertainty. Low complexity work is straightforward and well-understood; complicated work is technical but predictable; complex work is uncertain and unpredictable; chaotic work is unstable and needs to be tamed before any method is applied. Agile is for complex work: consider designing a new product with new tech or running a marketing campaign with shifting customer preferences.

When requirements are certain and technology is understood—like constructing a house or processing payroll—predictive methods (waterfall) may be more productive. Agile also doesn't work well when work can't be broken down into modules or when feedback from customers can't be obtained quickly.

## 1.3 Common Pitfalls in Agile Transformations

The presentation points out a number of ways in which organisations go wrong with agile:

1. Big bang changes: Changing thousands of employees to squads overnight is confusing and traumatic. Agile change is incremental.
2. Copy and paste models: Copying the Spotify tribe and guild structure without understanding their culture leads to disaster.
3. Using agile as a euphemism for layoffs: Using agile as a cover for downsizing erodes trust.
4. Top-down micromanagement: Managers who impose agile but still make all the decisions themselves are anti-agile.

Real change takes time, learning, and leadership support. Agile doesn't replace management; it redefines it to include more coaching, removing obstacles, and aligning teams with strategy.

## 1.4 Balancing Bureaucracy and Innovation

Agile is not anti-bureaucracy. Bureaucratic processes are necessary for repeatable, high-risk tasks such as safety, accounting, and legal issues. Agile is a complement to these processes that enables innovation. A balanced enterprise maintains operational discipline and introduces opportunities for cross-functional teams to innovate, learn, and rapidly change direction. It is up to the leadership to determine where structure is beneficial and where it is detrimental, with the aim of not eliminating hierarchy but finding the minimum viable bureaucracy to facilitate innovation.

**WHY AGILE FAILS**

https://www.scrum.org/resources/blog/why-agile-fails-mindset-shift-organisations-miss

https://medium.com/@internationalagilefederation/10-agile-misconceptions-lets-make-them-right-a628d43f85a7

# 2 Project Life-Cycle Models and Complexity

https://www.ituonline.com/blogs/project-life-cycles-predictive-iterative-incremental-and-adaptive/

## 2.1 Definable vs Exploratory Work

Projects range from definable to exploratory. Definable work employs well-known processes, such as assembling consumer products or processing insurance claims. Exploratory work entails solving new problems, like creating a new IoT logistics solution. In this case, the team must learn and adapt. As automation takes over more routine work, the amount of exploratory work increases.

## 2.2 High Uncertainty and the Stacey Model

The Stacey model assists in determining whether to apply predictive or adaptive methods. When product specifications, technology, or processes are uncertain, uncertainty and risk are high. Agile reduces risk by providing value in small increments, rapidly testing and incorporating feedback.

## 2.3 Life-Cycle Options

There are four types of life-cycle models:

1. Predictive (Waterfall) – detailed planning and phase-by-phase development are best when requirements and technology are mature.
2. Iterative – prototypes or proof-of-concepts are developed to test and refine requirements; this is appropriate for complex problems.
3. Incremental – subsets of the product are developed and delivered early to provide immediate value.
4. Agile – combines iterative and incremental development, assuming change is inevitable and focusing on continuous planning and reprioritization.

Agile contracts focus on collaboration and flexibility, in contrast to traditional contracts that depend on fixed scope and can result in contentious negotiations when changes are requested.

### 2.4 The Agile Manifesto

The Agile Manifesto emphasizes individuals and interactions, working software, customer collaboration, and responding to change over processes, documentation, contracts, and plans. There are twelve principles for agile teams: customers are delighted by early and continuous delivery, changing requirements are encouraged, working product is delivered frequently, business and developers work together daily, projects are built around motivated individuals, face-to-face communication is preferred, progress is measured by working software, a sustainable pace is maintained, excellence and simplicity in technology are sought, self-organizing teams are formed, and improvements are made through regular reflection.

# 3 Scrum Basics

https://scrumguides.org/scrum-guide.html

### 3.1 Scrum at a Glance

Scrum is a simple framework for teams to produce value by working towards an objective through iteration. Scrum was created by Jeff Sutherland and Ken Schwaber and is based on empiricism and the principles of lean. The three legs of Scrum are transparency, inspection, and adaptation. Scrum emphasizes commitment, focus, openness, respect, and courage.

### 3.2 Roles and Responsibilities

A Scrum Team consists of ten or fewer members and has one Product Owner (PO), one Scrum Master (SM), and a set of Developers. Developers plan work, develop usable product, and adhere to the Definition of Done. The Product Owner maximizes value by creating the Product Goal, prioritizing the product backlog, and communicating priorities to stakeholders. The Scrum Master serves as a guide for the team, impediments are removed, and the Scrum Master leads the organization in adopting Scrum.

### 3.3 Events

Scrum breaks down work into Sprints that are a maximum of one month. The Sprint begins with the Sprint Planning event, where the team commits to a Sprint Goal and chooses items from the product backlog to work on. The developers then have a Daily Scrum, where they inspect their work and adjust their plans. At the end of the Sprint, the team conducts a Sprint Review with the stakeholders to review the work done. Finally, the team conducts a Sprint Retrospective.

### 3.4 Artifacts and Commitments

There are three artifacts in Scrum. The Product Backlog is an ordered list of all work needed to improve the product, and it is associated with a Product Goal that has a longer-term focus. The Sprint Backlog holds the Sprint Goal, chosen items from the Product Backlog, and a plan for implementation. An Increment is the sum of all work completed from the Product Backlog that meets the Definition of Done. Each artifact is associated with a commitment: the Product Goal, Sprint Goal, and Definition of Done.

# 4 Kanban Essentials

https://kanban.university/kanban-guide/#kanban-method

https://www.atlassian.com/agile/kanban

### 1. Meaning of Kanban

Kanban comes from Japanese words **“kan” (visual)** and **“ban” (card/board)** and refers to a visual signaling system used to manage and control workflow.

### 2. Kanban System Purpose

A Kanban system uses visual tokens/cards to **manage and optimize the flow of work**, originally inspired by supermarket inventory replenishment.

### 3. Pull-Based Workflow

Kanban creates a **pull system**, meaning new work is started only when capacity is available, preventing overloading and improving flow.

**4. Six Core Practices of Kanban**

The method focuses on:

- Visualize work
- Limit Work in Progress (WIP)
- Manage flow
- Make policies explicit
- Implement feedback loops
- Improve collaboratively and evolve experimentally

**5. Key Flow Metrics**

Two primary performance measures are:

**Cycle Time:** Time from start of work to completion

**Lead Time:** Time from request to delivery
Lead time includes queue time plus cycle time.

**6. Impact of WIP Limits**

Reducing Work in Progress (WIP) **improves throughput and reduces cycle time**, leading to smoother and faster delivery.

**7. Role of Cumulative Flow Diagram (CFD)**

CFD visualizes system health by showing relationships among **lead time, cycle time, backlog, and WIP**, helping teams detect bottlenecks and flow issues.

### 4.1 Pull Systems and Visualization

Kanban is a method and a mindset that was first developed in the lean manufacturing sector. Kanban uses boards and cards to visualize the work flow. Kanban is different from push systems, which initiate new work regardless of capacity. Kanban is a pull system, which means that teams initiate new work only when there is capacity for it. Limiting Work in Process (WIP) increases efficiency and reduces cycle time.

### 4.2 Core Practices

There are six core practices that Kanban promotes. These include visualization of work, limiting WIP, managing flow, making policies explicit, using feedback loops, and collaborative improvement. Two important metrics are used to measure the performance of Kanban teams. These are cycle time, which is the time it takes to complete work once it has started, and lead time, which is the time from receipt of request to delivery. A cumulative flow diagram is used to illustrate the relationship between the backlog, WIP, and completed work, which helps identify bottlenecks.

Kanban can be used in conjunction with Scrum by implementing visualization and WIP limits for Sprints. Scrumban is a popular approach that combines Scrum events with Kanban flow.

# 5 Portfolio and Program Management

https://www.onethreadapp.com/blog/traditional-vs-agile-project-management/#elementor-toc__heading-anchor-3

### 5.1 Traditional Portfolio Management

In traditional project portfolio management (PPM), budgets are assigned annually through thorough business cases and financial analysis. Projects go through stage-and-gate reviews, and success is determined by scope, schedule, and budget. The hierarchical nature of PPM makes it challenging to make changes in the middle of the cycle and makes experimentation hard.

### 5.2 Agile Portfolio Management

In agile portfolio management (APM), value streams are funded instead of traditional projects, and rolling-wave planning is used instead of annual budgeting. Prioritization of work is done continuously based on strategic goals and customer feedback. Sessions such as portfolio syncs, strategic reviews, and retrospectives facilitate alignment between leadership and teams. Artifacts used in APM include a portfolio backlog, vision, roadmaps, and user story hierarchies, while metrics measure product outcomes, process, and culture.

To adopt APM, one must evaluate readiness, select a scaling approach, provide training for leadership and teams, and overcome cultural barriers. Organizations must adapt governance to their size and regulatory requirements; there is no one-size-fits-all approach.

# 6 Scaling Frameworks

### 6.1 Scaling from a Single Scrum Team

**https://www.scrumatscale.com/scrum-at-scale-guide-online/**

When a single Scrum team is insufficient, scaling frameworks are employed. Scaling frameworks apply agile concepts to multiple teams and organizational departments, as well as deal with issues of coordination, governance, and strategy. The following sections describe four popular scaling frameworks: Scrum@Scale, Large-Scale Scrum (LeSS), Disciplined Agile (DA), and the Scaled Agile Framework (SAFe). Additional information from other sources is also included.

Scrum@Scale is an approach developed by Jeff Sutherland to scale Scrum over a network of teams. It is designed to separate concerns, where the Scrum Master Cycle deals with the coordination of the work, and the Product Owner Cycle deals with the alignment of the build. The important events in this framework are Scrum of Scrums, where representatives from different teams coordinate impediments and integration, Scaled Daily Scrums, Scaled Retrospectives, Executive Action Teams (EAT), and Executive MetaScrums (EMS).

Why scale Scrum? As organizations increase in size, no single team can manage the complexity or amount of work; scaling Scrum helps ensure improved teamwork, consistency, speed, and alignment with business objectives. Scrum@Scale maintains the values of Scrum—flexibility, transparency, and efficiency—but adds only the necessary amount of coordination to prevent chaos. The approach outlines a minimum viable bureaucracy, requiring only the roles and meetings required to keep teams in sync.

Some recent studies have pointed out that Scrum@Scale is a scale-free architecture, where the same patterns are repeated at various scales, allowing for organic growth without the need for restructuring. The Product Owner Cycle (with a Chief Product Owner and MetaScrum backlog) helps keep multiple teams in sync with strategic objectives, while the Scrum Master Cycle helps ensure smooth execution. Since it is based on existing Scrum processes, organizations already using Scrum can easily adopt Scrum@Scale with little retraining. However, to be successful, organizations need to ensure that their Scrum processes are sound at the foundation level; teams need to be disciplined about backlog development, Definition of Done, and cross-team integration.

One of the key attractions of Scrum@Scale is its flexibility. Its network architecture enables teams to be rearranged as business needs dictate, and organizations can easily adopt Scrum@Scale incrementally. It is widely used in technology companies and digital product organizations that value speed and innovation. However, organizations in industries with strict compliance needs, such as finance or healthcare, may find Scrum@Scale's minimal documentation requirements inadequate.

### 6.2 Large-Scale Scrum (LeSS) – Scrum for Multiple Teams

**https://less.works/less/framework**

LeSS is an application of Scrum principles to multiple teams working on a single product. It has two forms: LeSS (for two to eight teams) and LeSS Huge (for eight or more teams). In LeSS, all teams have one Product Owner, one product backlog, and one Sprint. The events are a joint sprint planning (a combined phase to split the backlog items, then team planning), multiple team product backlog refinements, and an overall retrospective. LeSS promotes system thinking. Teams are encouraged to optimize the system, not the local optimum.

LeSS scales by removing management tiers instead of adding them. LeSS Huge has up to eight feature teams with a shared Product Owner. Area Product Owners oversee backlogs for domains and report to the overall Product Owner. This makes it easy to coordinate and align. However, LeSS requires a paradigm shift. Hierarchical thinking is replaced by servant leadership. Teams must increase their knowledge of Scrum basics. Training is aimed at increasing Scrum knowledge rather than learning new activities.

The flexibility of LeSS enables teams to organize themselves for customer value. Since LeSS has very few constraints, teams are encouraged to try and experiment. LeSS is suitable for product-oriented companies with robust engineering cultures and a single product or platform. It is not suitable for companies that need strict compliance or for companies with multiple unrelated product lines. Removing organizational tiers may face resistance. Executive support is essential for success.

### 6.3 Disciplined Agile (DA) – A Toolkit for Tailoring Your Way of Working

**Foundation for Business Agility | Disciplined Agile®**

Disciplined Agile was created as a decision tool, not a prescriptive approach. The DA mantra is "choose your way of working" (WoW), acknowledging that there is no one-size-fits-all solution. DA is based on agile, lean, Theory of Constraints, and organizational development and recommends that teams choose and improve their life cycle from a variety of options including agile, lean, continuous delivery, and exploratory lifecycles.

**In Disciplined Agile, teams select an appropriate lifecycle model based on contextual factors and progressively evolve their Way of Working (WoW) through continuous improvement.**

In contrast to prescriptive models, DA focuses on enterprise awareness. It encourages teams to remember that they are part of a broader ecosystem and need to think about architecture, data management, security, and governance from the outset. DA enables scaling along multiple axes: team size, project size, geography, and regulatory complexity. DA is especially useful for enterprises with multiple projects and a combination of new and legacy systems, as it offers advice without imposing consistency.

The application of DA can be done in a phased manner. Teams begin with simple agile methods and introduce more advanced methods as they develop. The fact that DA is centered on decision-making makes continuous learning and coaching essential. Enterprises require seasoned coaches to guide teams through decision-making; the focus is on building judgment, not on memorizing steps. DA is appropriate for large and complex enterprises with robust change management processes. Startups and small businesses may find it too cumbersome.

Purpose and definition: Scrum@Scale seeks linear scalability by ensuring that the growth of a network of teams is not hampered by unnecessary processes. The guide describes a minimum viable bureaucracy as the "smallest amount of governance and processes required to support the functions of an organisation without hindering the creation of customer value." Reducing decision latency, or the time taken to make a decision, is cited as one of the success factors.

Organisational structure: A Scrum of Scrums (SoS) requires two leadership teams: the Executive MetaScrum (EMS) and the Executive Action Team (EAT). EMS is concerned with what is produced, while EAT is concerned with how it is produced. This helps the network of teams to coordinate without becoming too hierarchical.

Events:

Scaled Daily Scrum (SDS): A 15-minute meeting where representatives from each team discuss overall progress, problems, and inter-team dependencies. The idea is to coordinate plans and eliminate any obstacles that might impact any team's Sprint Goal.

Scaled Retrospective: This event occurs every Sprint and involves all Scrum Masters in the network of teams to share experiences of improvement experiments and how they can be applied across teams.

Executive roles: EAT is responsible for ensuring that Scrum roles are established and supported, events occur, artifacts are transparent, and organisation-wide impediments are removed. EMS is a strategic meeting point for the Chief Product Owners and executives, who discuss vision and priorities, negotiate budgets, and align all teams to a common backlog.

Disciplined Agile (DA)

Definition: DA is a toolkit that combines the theories of Flow, Lean, Theory of Constraints, and organizational development to offer simple, direct advice on how to make better decisions about your "way of working" (WoW). It is an umbrella over all agile methods and helps to explain when to apply which technique.

Aims and benefits: DA helps organizations to scale agility from teams to the whole enterprise. It helps to speed up value delivery in scaled agile environments by realizing that scaling is more than just scaling team size. Most importantly, DA strongly states that "one way of working for all teams will not suffice" and that teams should be free to choose and improve their WoW to adapt to their own and ever-changing situations.

Mindset and framework: DA mindset looks at people, processes, and flow, covering roles, responsibilities, life-cycles, workflows, and goal diagrams. Teams are encouraged to choose a life-

cycle that fits their context. These life-cycles support Agile, Lean, Exploratory, Continuous Delivery (Agile or Lean), and Program life-cycles. These life-cycles are an extension of Scrum, DevOps, Kanban, and Lean Startup patterns in the organization.

Governance: DA uses lean governance with milestone questions to ensure that the stakeholders have vision, valid architecture, continued viability, enough functionality, production readiness, and delighted stakeholders. This helps to encourage informed decision-making without prescribing a single process.

Organizations can scale agile successfully by understanding the role and events of Minimum Viable Bureaucracy and the flexible and context-driven approach of Disciplined Agile.

### 6.4 Scaled Agile Framework (SAFe) – Structure and Governance for Large Enterprises

**Framework - Scaled Agile Framework**

Enterprise --- Portfolio—ART—Teams

SAFe is one of the most popular scaling frameworks for large enterprises: Team, Program, Large Solution, and Portfolio. Each level has defined roles, ceremonies, and tools, making it a hierarchical system that aligns executives with individual teams. This makes it attractive to enterprises that are used to having a clear chain of command.

The scalability of SAFe is based on its planning processes. The Program Increment (PI) planning involves up to 150 people to align on priorities and dependencies for the next 8–12 weeks. The Agile Release Trains (ARTs) involve 50–125 people with 5–12 teams; multiple ARTs are synchronised at the Large Solution level to implement complex initiatives. A robust governance structure ensures that the portfolios provide strategic guidance, while programs provide tactical guidance.

SAFe provides comprehensive solutions for compliance, financial planning, and risk management. However, the complexity of SAFe can be overwhelming. SAFe provides more than 100 roles, events, and tools. The implementation of SAFe requires 12–18 months of training and coaching.. Critics argue that the amount of ceremony can impede teams and reduce autonomy. However, the trade-off is that it provides a mature governance structure that manages cross-team dependencies and compliance, which is essential in industries such as finance, healthcare, and aerospace.

SAFe provides some degree of flexibility. The SAFe Implementation Roadmap encourages teams to customize practices according to their needs and to inspect and adapt regularly. SAFe is suitable for large enterprises with 500 or more employees and strong project management culture. SAFe may not be suitable for small organizations or startups.

- SAFe Picture describes the process by which an organization transforms strategy into customer value using congruent layers of work. At the highest level, there is the Portfolio level which is the setting of strategy and funding priorities, here the portfolio backlog consists of epics that are organized around value streams to inform investment. In very complicated products, the Large Solution level links several Agile Release Trains (ARTs) and suppliers. The ART level is the primary delivery engine, a team of teams (approximately 50-125 individuals) of Business Owners, Product Management, System Architect, RTE team, and Agile teams collaborate to create increments of solution. At the Team level, we have Scrum or Kanban teams that develop stories which are aggregated into features and finally customer value. The whole movement is powered by the Continuous Delivery Pipeline (Continuous

Exploration, Integration, and Deployment), which allows transferring ideas to production swiftly and reliably and provides the customer with a continuous stream of feedback and value

**Leadership & Culture (circle + culture pyramid)**

A) Leadership behaviors (circle)

- Vision: clear direction
- Courage: willingness to change/innovate
- Growth: develop leaders/people
- Authenticity: lead by example
- Center: Continuous learning

**B) Culture pyramid (Beliefs → Values → Practices)**

- Beliefs shape values; values shape practices.
- If leaders want agile practices, they must change underlying beliefs and values too.
- Exam line: SAFe transformation is primarily a leadership/culture change, not a process change.

**Team & Technical Agility (PI/Iteration diagram)**

This shows how teams deliver within an ART cadence.

Key structure

- PI (Program Increment) = timebox of multiple iterations (often 8–12 weeks)
- Inside PI: multiple iterations + IP iteration (Innovation & Planning)

Key concepts shown

- PI Planning (start): align teams to objectives, plan features, identify dependencies
- Team backlogs: stories to implement
- Features & enablers:
- Feature = customer value
- Enabler = technical work that supports features (architecture, infrastructure)
- System demos: demonstrate integrated progress regularly
- Inspect & Adapt (end): evaluate outcomes, solve systemic problems
- DevOps + Built-in Quality
- Emphasis on quality practices to prevent “integration hell.”

**Product Development Flow (value flow loop)**

- This diagram shows a loop:
- Innovation → Vision → Strategy → Customer-centric design → Continuous delivery → Release on demand → Customer value → Feedback
- Key messages:
- Flow must be continuous, not batchy.
- Feedback is not after release only; it's continuous and drives next decisions.
- Release on demand = capability to release when needed (not forced by calendar).

**Large Solution Integration & Delivery (big system diagram)**

Used when:

- Multiple ARTs + suppliers build parts of a large solution.
- Need strong integration + governance.
- Key elements in the diagram:
- Solution intent/specs: shared understanding of what to build
- Pre-plan + coordinate and deliver
- Value streams across multiple ARTs/suppliers
- Solution demo: integrated demo at solution level
- Inspect & Adapt: improve system-level delivery
- Output: Integrated solution in a real solution context
- Exam line: This level manages integration complexity beyond a single ART.

**Lean Portfolio Management (LPM diagram)**

This is how SAFe manages strategy + funding.

Inputs

- Strategic themes / OKRs
- Portfolio vision
- Roadmap
- Budget
- Portfolio Kanban (funnel → done)
- Ideas/epics move through stages like:
- Funnel → Reviewing → Analyzing → Ready → Implementing → Done
- Value stream funding
- Funding is aligned to value streams, not random projects.
- Uses guardrails (spending boundaries, policies).

**Governance & measurement**

- Lean governance, KPIs, portfolio performance
- Often supported by a VMO style function (lightweight governance)
- Exam line: LPM aligns investments to strategy and funds value streams to improve flow.

What OKRs are (simple)

- Objective = the goal (qualitative, directional)
- Key Results = measurable outcomes proving the goal is achieved
- OKRs → drive strategy → generate Epics → broken into Features → broken into Stories

7 Key Takeaways

- Apply the Stacey matrix to determine whether your project is agile or predictive. Agile is suitable for complex and uncertain problems that can be broken down into modular components, while predictive methods are suitable for simple and stable tasks.
- Apply the agile mindset: emphasize continuous learning, experimentation, and customer collaboration. Resist big-bang transformations and customize practices to suit your culture.
- Understand the basic mechanics of Scrum: roles (PO, SM, Developers), events (Sprint, Sprint Planning, Daily Scrum, Review, Retrospective), and artifacts (Product Backlog, Sprint Backlog, Increment). Understand time-boxing and the purpose of each event.
- Understand Kanban metrics and techniques: visualization, work in progress, cycle time, lead time, cumulative flow diagrams.
- Appreciate the differences between life-cycle models: predictive, iterative, incremental, and agile. Predictive is best for stable requirements; agile integrates incremental, iterative, and feedback-based approaches.
- Understand the differences between portfolio management methods: Traditional PPM focuses on fixed scope, schedule, and budget; APM invests in value streams and rolling-wave planning.

Compare scaling methods:

- Scrum@Scale promotes minimal bureaucracy and fractal scaling, based on two cycles and scale-free architecture.
- LeSS reduces coordination overhead by eliminating management tiers and using one product backlog and one Product Owner for up to eight teams.
- Disciplined Agile is a flexible toolkit that helps teams pick and adapt their own way of working, with a focus on enterprise awareness and coaching.
- SAFe uses a hierarchical framework with Program Increments and Agile Release Trains, which is best suited for large, highly regulated enterprises but is difficult to implement.

By committing these frameworks and guidelines to memory, you'll be ready to answer exam questions that assess your knowledge of agile methods and their application at scale.

## Lean principles

1. Eliminate waste
2. Decide as late as possible
3. Deliver as fast as possible
4. empower the team
5. amplify learning
6. build integrity